# WEZTERM CONCEPTUAL ARCHITECTURE REPORT

PRESENTATION:
HTTPS://YOUTU.BE/KL4GGK6JI-M

By the Wezards of Oz

# TEAM MEMBER BREAKDOWN

**The report responsibilities were divided equally among members based on their strengths and preferences. Proof reading was done collectively and frequent in person meetings were held to ensure all members were informed on progress and direction.**

**Team Lead:** Emili Ribakov [*emili.ribakov@queensu.ca*]

**Presenter:** Ethan Rose [*23nfd5@queensu.ca*]

**Presenter:** Alyssa Pascal [*26ln26@queensu.ca*]

Jonah Schwartz [*jonah.schwartz@queensu.ca*]

Emilia Markovic [*23jf44@queensu.ca*]

Addison Lovegrove [*23ppb3@queensu.ca*]

# SYSTEM FUNCTION

Cross-platform terminal emulator and multiplexer

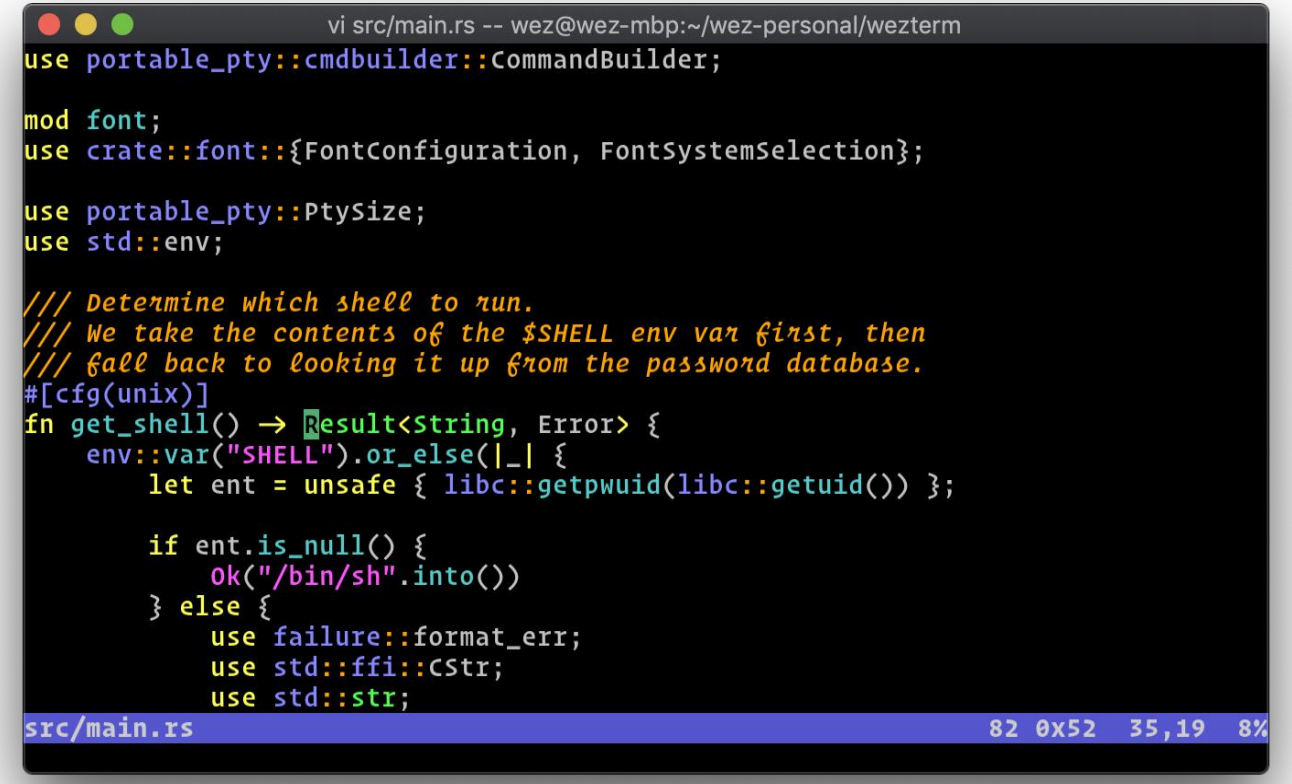
Written in Rust

Two-way bridge between the user and shell

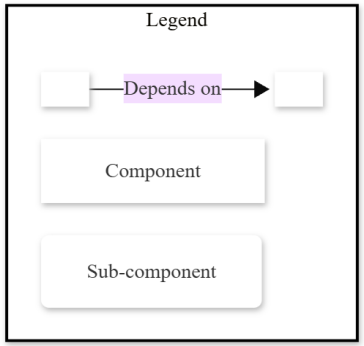
Multiplexed windows, tabs, and panes

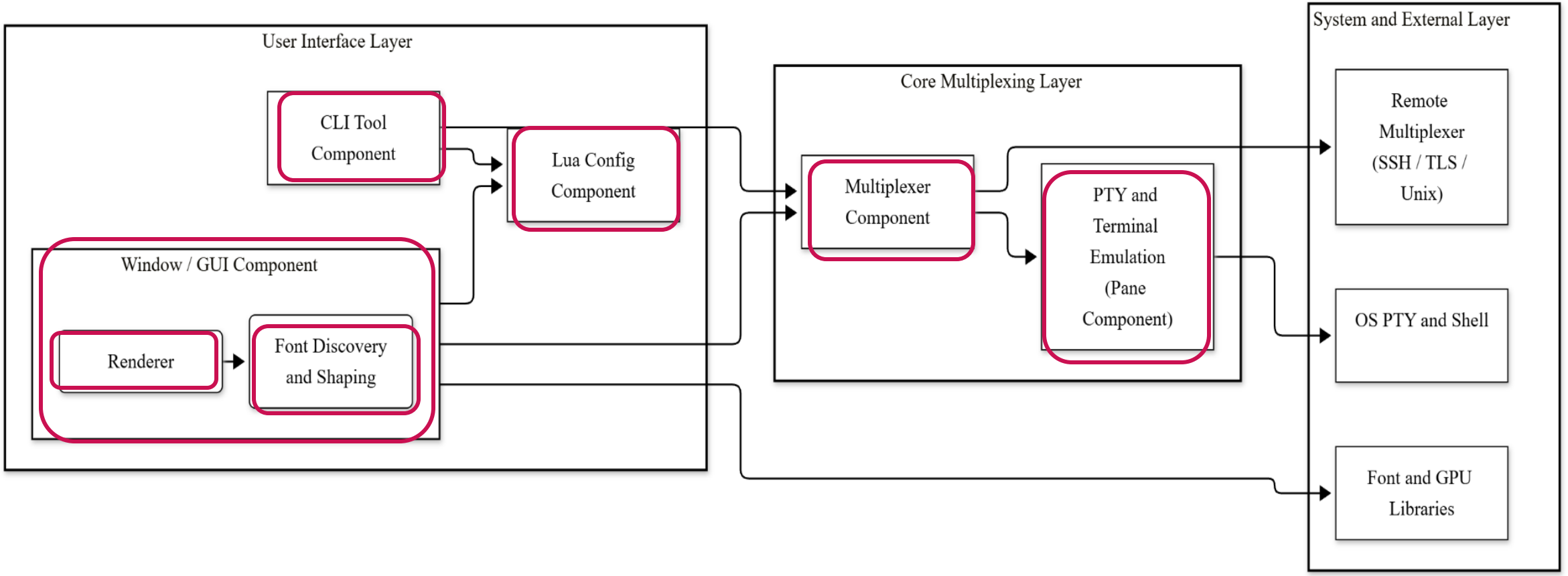
GPU accelerated text rendering

Programmable configuration in Lua

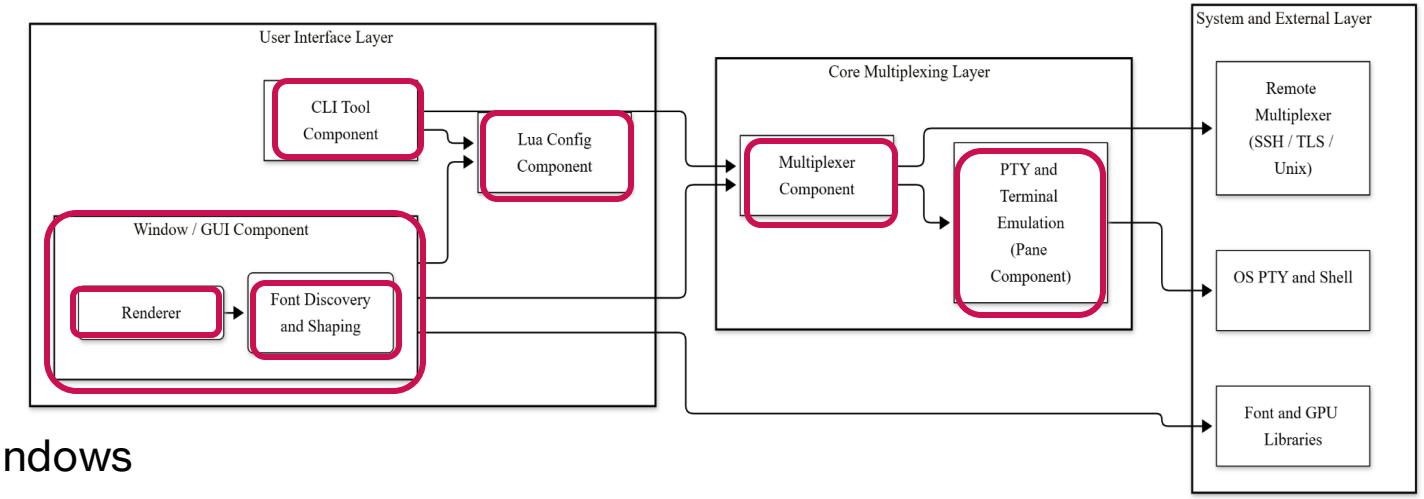


# Components in WezTerm



# PARTS AND INTERACTIONS




**Window/GUI component:** receives input, manages windows and tabs

**Renderer:** draws terminal content, caching glyphs

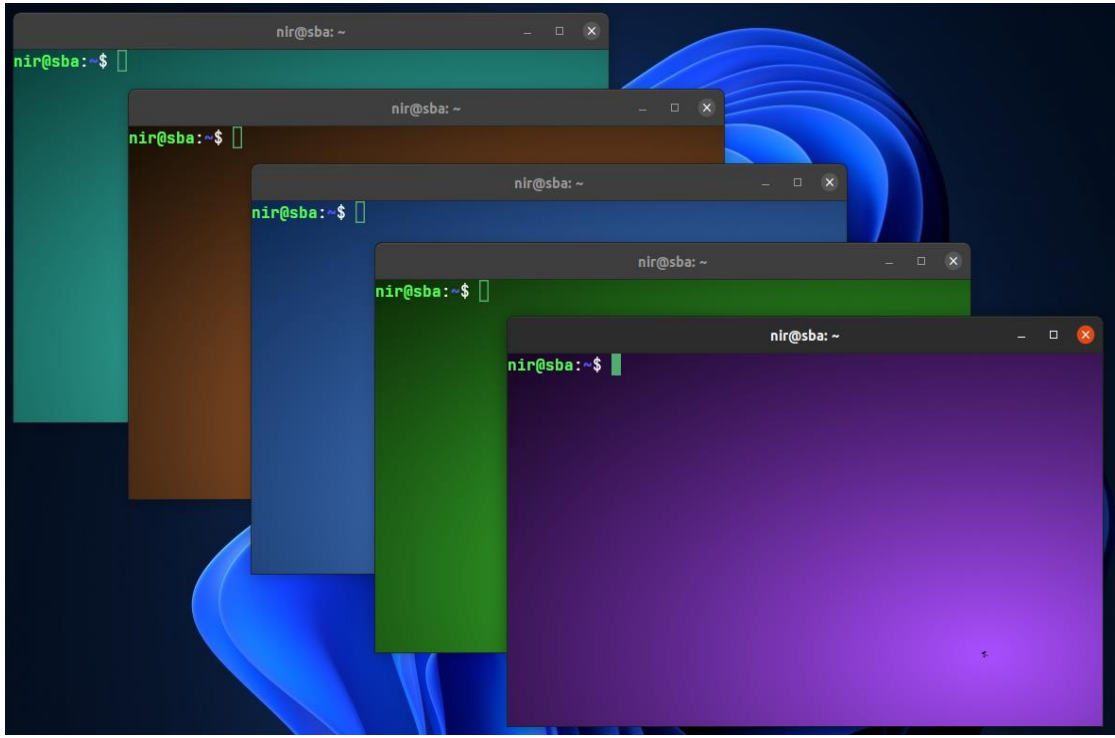
**Font discovery and shaping:** finds fonts and shapes glyphs (HarfBuzz)

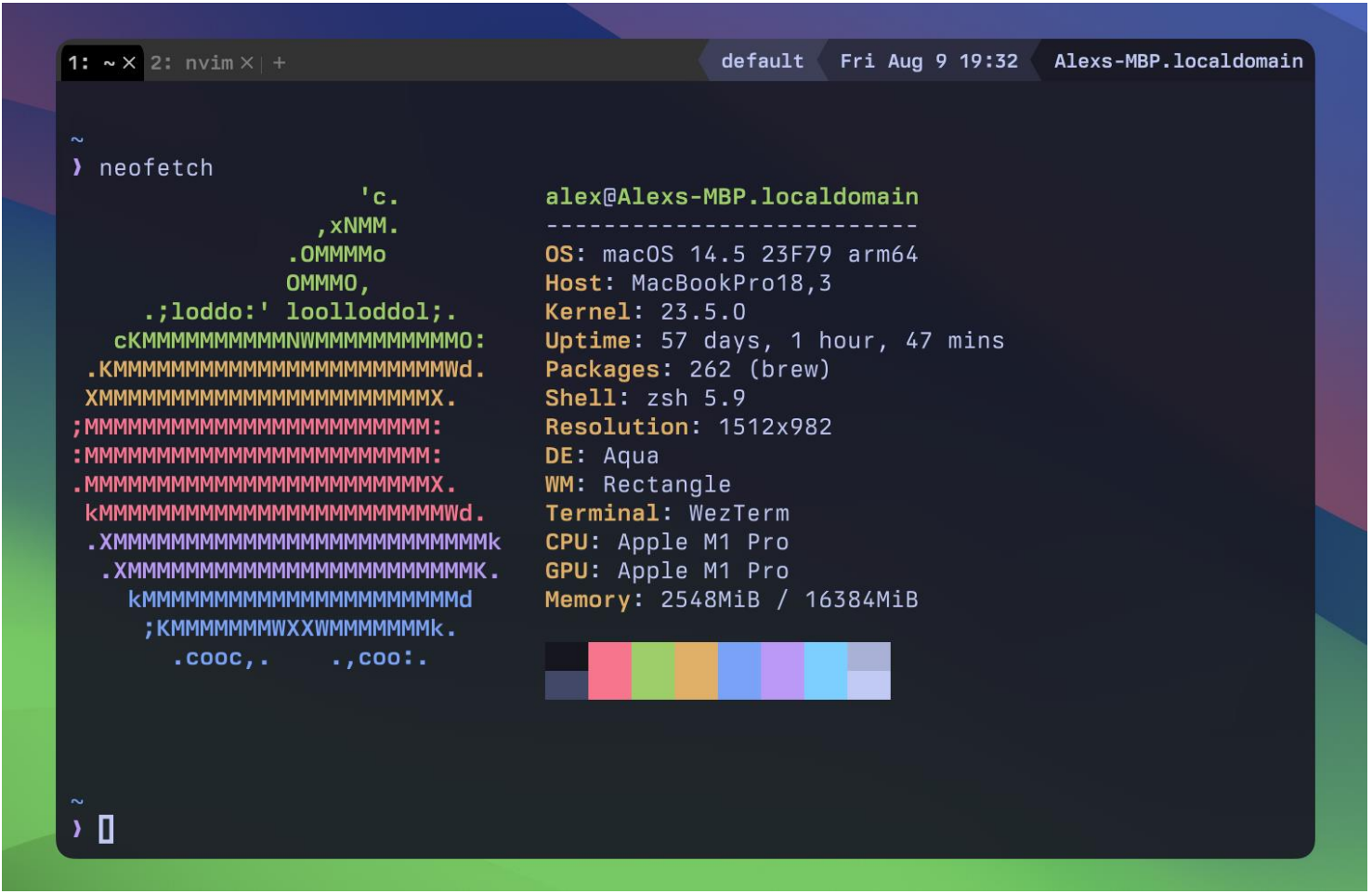
**Lua Config engine:** bindings, fonts, domains customization through Lua

**Command-line interface(CLI):** launches and scripts WezTerm

**Multiplexer:** owns domains, windows, tabs, and panes

**PTY and Terminal emulation(Pane)**: connects to the OS PTY and interprets output into terminal state.






# EVOLVABILITY

Swappable renderers: OpenGL, WebGPU, (CPU-based)Software

New domain types plug in: local, SSH, TLS and Unix domains

Lua config extends without code changes

HarfBuzz handles shaping correctness

Risk: renderer changes

Risk: Lua and CLI interfaces become commitments to users

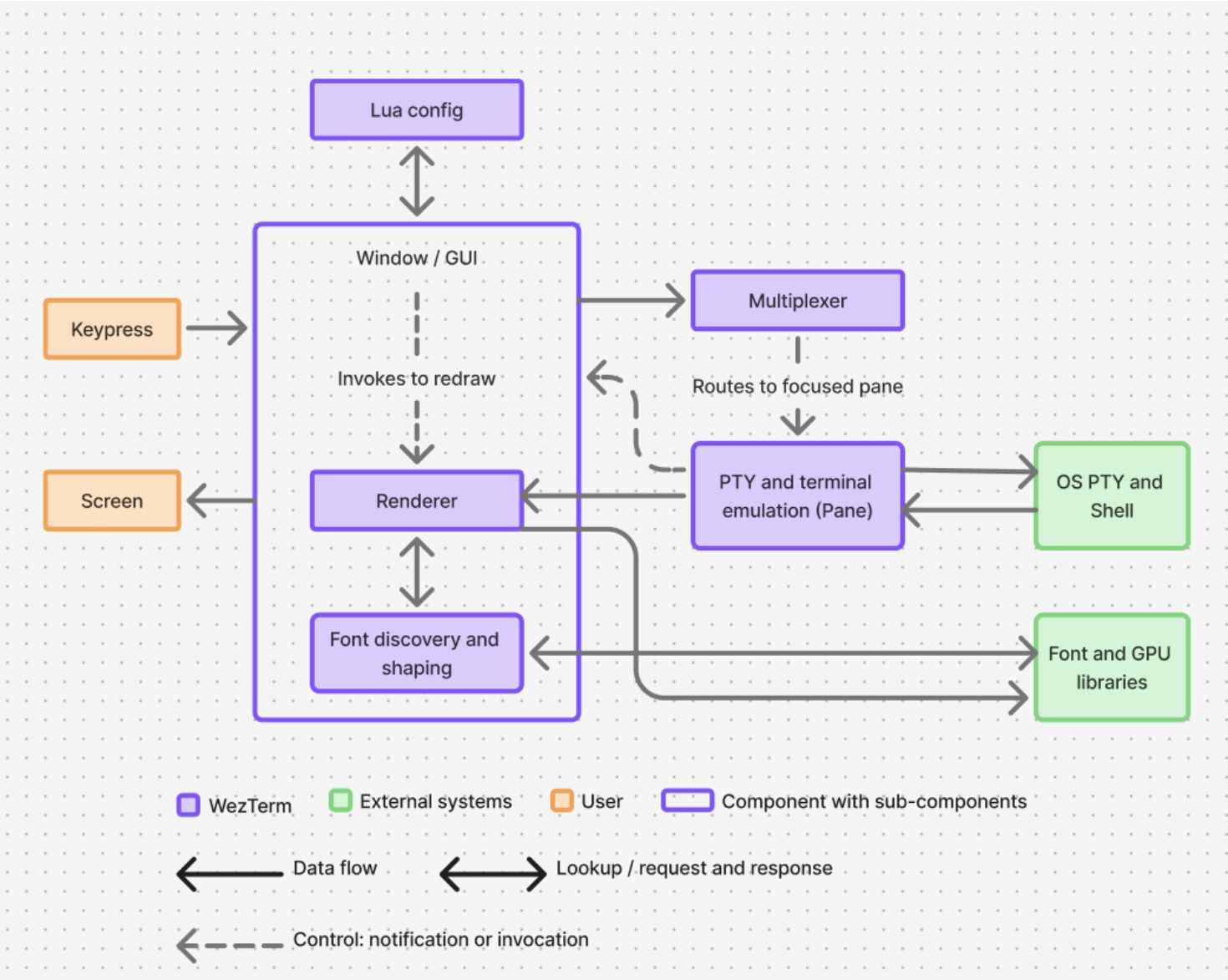
# CONTROL AND DATA FLOW

**Event-driven control flow:** components react to events, not a fixed order

**Input:** Keypress → GUI (checks Lua bindings) → Mux → Pane → PTY → Shell

**Output:** Shell → PTY → Pane (interprets into terminal state) → Renderer (+ fonts) → Screen

Direct calls within a process; messages across boundaries (PTY, sockets, SSH)



# DIVIDING RESPONSIBILITIES AND THE IMPLICATIONS

Mux and networking

Terminal emulation

Renderer and fonts

GUI and per-OS windowing

Lua config and CLI

Shared risk: mux model, renderer testing



# **USE CASE:** RESUME A REMOTE DEVELOPMENT SESSION

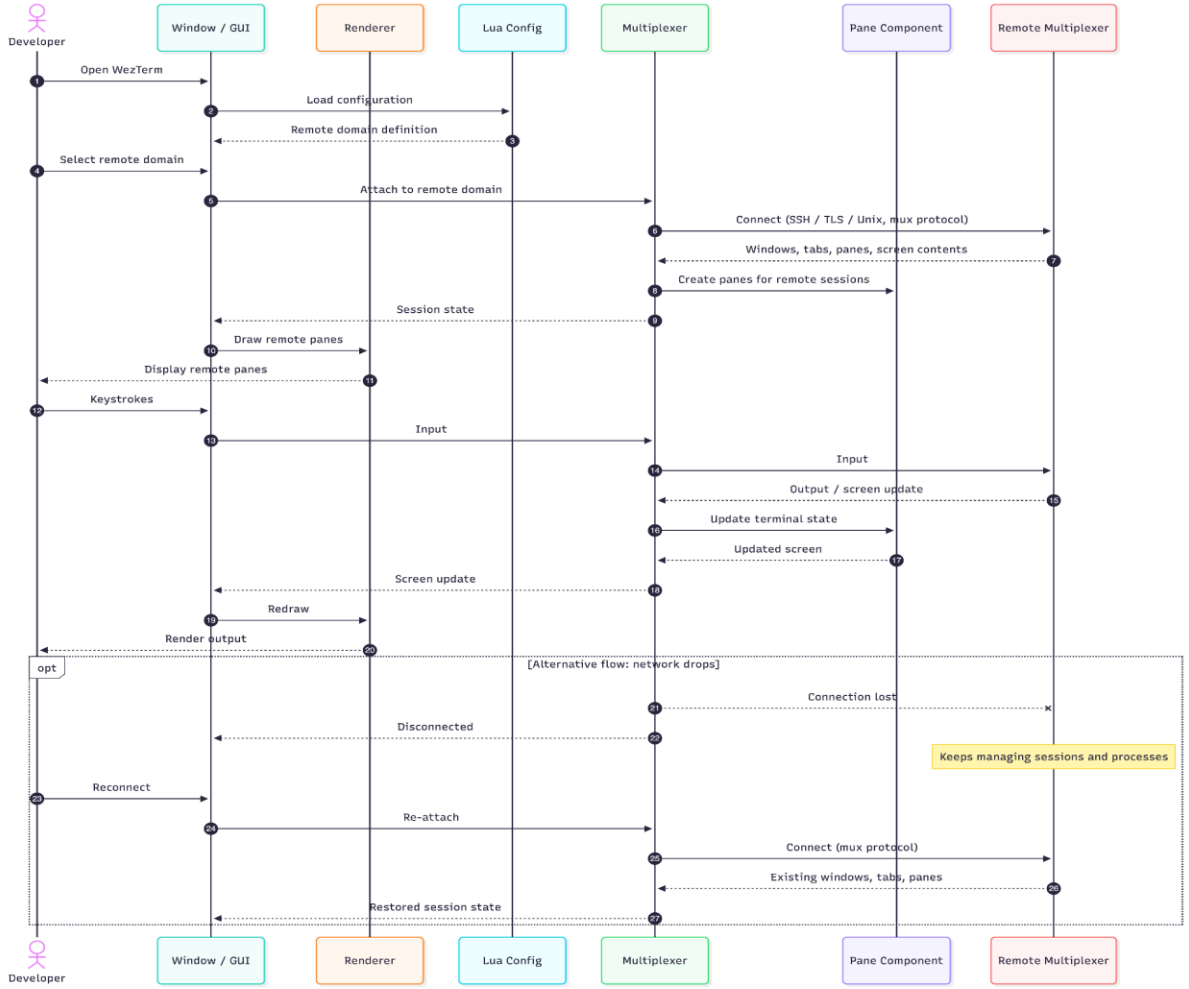
WezTerm loads config defining remote domain

Domain manager connects via transport layer

Mux server sends windows, tabs, panes

Disconnection leaves remote PTY processes running

Reconnecting restores sessions without restarting processes



# ARCHITECTURAL STYLES - LAYERED

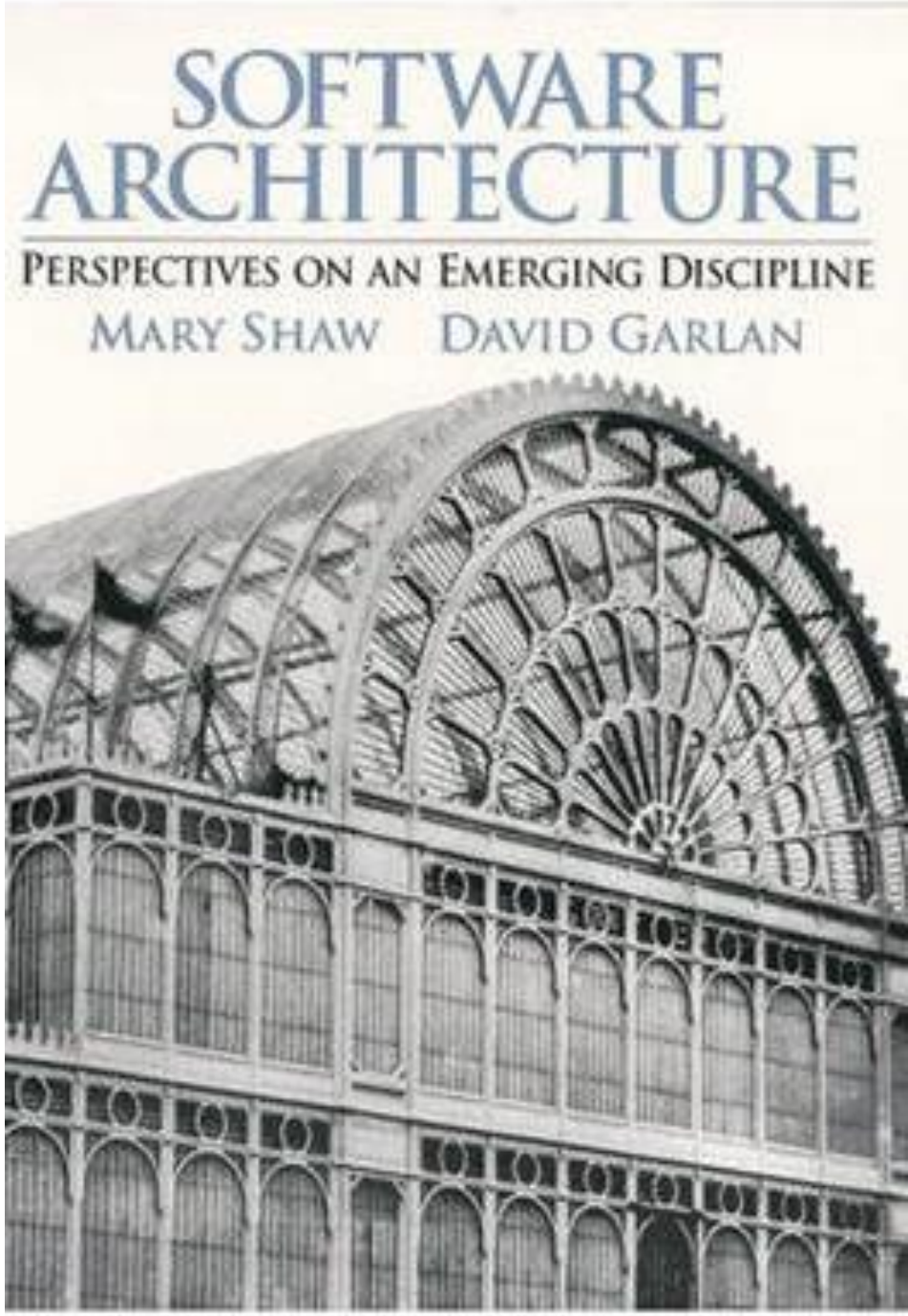
GUI → multiplexer → terminal model → PTY/OS

Benefit: GUI changes without touching sessions

Benefit: sessions survive without a window

Cost: not strict, so boundaries blur

Cost: extra hops between layers



# INTERACTION AND QUALITY CONTROL WITH AI

Interaction with AI teammate:

- Each member acted as their own prompt engineer
- Tasks: requirements and rubric checks, concept clarification, structure, diagram critique
- Prompts improved: vague requests → context + requirements + output formula

Quality Control:

- Facts checked against external sources
- Changes agreed with the section's author first
- Suggestions rejected when they didn't fit (e.g. kept our Lua use case)
- Estimated contribution: ~15%, mostly review and feedback



# CONCLUSION!

Software engineering large scale systems is hard!

They require teams of people coordinating together

WezTerm structure focuses on linking components smoothly

Understanding these projects require cross domain knowledge

Don't expect to understand everything at first

